

1. Convex shape

1. Convex sets

$C \subseteq \mathbb{R}^d$ is **convex** if, for every $x, y \in C$ and $\lambda \in [0, 1]$,

$$\lambda x + (1 - \lambda)y \in C.$$

To prove convexity, handle arbitrary x, y, λ . To refute it, one escaping segment point suffices.

For $p \geq 1$ and $r \geq 0$, $\{w : \|w\|_p \leq r\}$ is convex:

$$\|\lambda x + (1 - \lambda)y\|_p \leq \lambda\|x\|_p + (1 - \lambda)\|y\|_p \leq r.$$

The annulus $\{w \in \mathbb{R}^2 : 1 \leq \|w\|_2 \leq 2\}$ is not convex: $(1, 0)$ and $(-1, 0)$ are feasible, but their midpoint $(0, 0)$ is not.

2. Convex functions

On a convex set C , $f : C \rightarrow \mathbb{R}$ is **convex** if, for every $x, y \in C$ and $\lambda \in [0, 1]$,

$$f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda)f(y).$$

The graph lies below its endpoint chords. The convex-domain condition is part of the definition.

- **Set witness:** a segment leaves the feasible region.
- **Function witness:** a graph point rises above its chord.
- A violated midpoint ($\lambda = \frac{1}{2}$) is a complete counterexample.

Keep the properties separate: a convex set keeps interpolated parameters legal; a convex objective stays below chords. Neither supplies the other.

2. First-order reasoning

1. Tangent lower bound

Let $f : C \rightarrow \mathbb{R}$ be differentiable on an **open convex set** C . Then f is convex if and only if, for all $x, y \in C$,

$$f(y) \geq f(x) + \nabla f(x)^T(y - x).$$

Every tangent hyperplane is therefore a global lower bound.

If $\nabla f(w^*) = 0$, then for every y ,

$$f(y) \geq f(w^*) + 0^T(y - w^*) = f(w^*),$$

so w^* is a global minimizer. Convexity is essential; a zero gradient alone is insufficient for a nonconvex function.

2. Unconstrained gradient method

For differentiable $f : \mathbb{R}^d \rightarrow \mathbb{R}$, initial point w_0 , and positive step size η ,

$$w_{t+1} = w_t - \eta \nabla f(w_t).$$

One update requires w_t , the current gradient, and a declared η . The minus sign moves against the local direction of greatest increase. No projection onto a constraint set is included.

The rule is a state transition, not a convergence theorem: decrease and convergence depend on the objective and step size.

3. Quadratic step-size diagnosis

For $f(w) = \frac{1}{2}(w - 3)^2$, $f'(w) = w - 3$. Writing $e_t = w_t - 3$ gives

$$e_{t+1} = (1 - \eta)e_t.$$

η	$1 - \eta$	Behavior
$\frac{1}{2}$	$\frac{1}{2}$	same side; distance halves
$\frac{3}{2}$	$-\frac{1}{2}$	crosses; distance halves
2	-1	alternates; value does not fall
$\frac{5}{2}$	$-\frac{3}{2}$	alternates; distance grows

Overshoot alone is not the diagnosis; inspect $|1 - \eta|$.

3. Feasibility audits

1. Polynomial in encoded size

After fixing a computation model and encoding, an algorithm is **polynomial time** if its operation count is at most cN^k for instance-independent constants c, k , where N is encoded input length. State every parameter that scales.

For a binary integer M , $N \approx \log_2 M$; an M -step loop is exponential in encoded length because $M \approx 2^N$.

- Scan N stored values once: $O(N)$.
- Enumerate $\{0, 1\}^d$: 2^d evaluations.

- $O(d^3) + O(2^d d)$ is exponential in variable d .
 - If d is genuinely fixed, 2^d is constant relative to other inputs.
- Faster hardware changes constants, not the classification.

2. Computational–statistical gap

A **computational–statistical gap** occurs when sample guarantees permit the desired accuracy, but no computationally efficient method is known or available under the same assumptions.

Qualified positive case: under the one-way-function assumption, polynomially many labeled examples suffice for an inverse-function family, while a polynomial-time learner would provide an efficient inverter and contradict the assumption.

Showing that one exact implementation enumerates 2^d candidates proves only that the implementation is inefficient. Without a hardness result, it does not prove the task inherently hard; a structured algorithm may exist.

3. Three independent audits

Audit	Evidence	Do not infer
Statistical	bounded independent losses; Hoeffding	optimizer is fast
Optimization	segment/chord checks; tangent bound	data suffice
Computational	polynomial encoded-input runtime	output is accurate

Pitfalls:

- Dependence makes the displayed independent-loss Hoeffding claim unsupported.
- One infeasible midpoint refutes feasible-set convexity.
- Polynomial sample need does not make 2^d enumeration efficient.
- Undeclared η leaves gradient-update decrease or convergence unsupported.

A learning guarantee is only as strong as its weakest unproved link.